

Figure 1: Hardware mapping

Once mapped to virtual addresses, it depends on the device datasheet as to which set of device registers and/or device memory to read from or write into, by adding their offsets to the virtual address returned by `ioremap()`. For that, the following are the APIs (also prototyped in `<asm/io.h>`):

```
unsigned int ioread8(void *virt_addr);
unsigned int ioread16(void *virt_addr);
unsigned int ioread32(void *virt_addr);
unsigned int iowrite8(u8 value, void *virt_addr);
unsigned int iowrite16(u16 value, void *virt_addr);
unsigned int iowrite32(u32 value, void *virt_addr);
```

Accessing the video RAM of 'DOS' days

After this first set of information, students were directed for the live experiments. The suggested initial experiment was with the video RAM of 'DOS' days, to understand the usage of the above APIs. Shweta got onto the system and went through `/proc/iomem` (as in Figure 2) and got the video RAM address, ranging from `0x000A0000` to `0x000BFFFF`. She added the above APIs, with appropriate parameters, into the constructor and destructor of her existing 'null' driver, to convert it into a 'vram' driver. Then she added the user access to the video RAM through read and write calls of the 'vram' driver; here's her new file, `video_ram.c`:

```
#include <linux/module.h>
#include <linux/version.h>
#include <linux/kernel.h>
#include <linux/types.h>
#include <linux/kdev_t.h>
#include <linux/fs.h>
#include <linux/device.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>
#include <asm/io.h>

#define VRAM_BASE 0x000A0000
#define VRAM_SIZE 0x00020000

static void __iomem *vram;
static dev_t first;
static struct cdev c_dev;
```

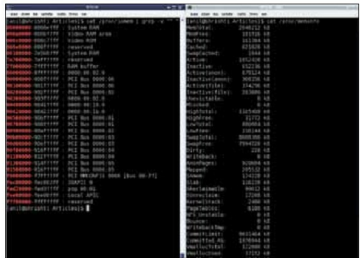


Figure 2: Physical and bus addresses on an x86 system

```
static struct class *cl;

static int my_open(struct inode *i, struct file *f)
{
    return 0;
}

static int my_close(struct inode *i, struct file *f)
{
    return 0;
}

static ssize_t my_read(struct file *f, char __user *buf,
size_t len, loff_t *off)
{
    int i;
    u8 byte;

    if (*off >= VRAM_SIZE)
    {
        return 0;
    }
    if (*off + len > VRAM_SIZE)
    {
        len = VRAM_SIZE - *off;
    }
    for (i = 0; i < len; i++)
    {
        byte = ioread8((u8 *)vram + *off + i);
        if (copy_to_user(buf + i, &byte, 1))
        {
            return -EFAULT;
        }
    }
    *off += len;

    return len;
}

static ssize_t my_write(struct file *f, const char __user
*buf, size_t len, loff_t *off)
```